

CLAIMIT

Code Analysis — Step by Step

A walkthrough of how the Claimit codebase actually works: how the backend boots, how the mobile app talks to it, and what happens — step by step, in the real code — for the two flows that matter most: signing in and filing a claim.

Based on the FastAPI backend ([fastapi/backend/app](#)) and the Flutter app ([claimitapporg/lib](#)) · 24 August 2026

Contents

1. Codebase map	3
2. Step by step — backend startup (main.py, database.py)	4
3. Step by step — how the app talks to the backend (ApiClient)	5
4. Step by step — sign in / OTP flow, end to end	6
5. Step by step — filing a claim, end to end	7
6. Database design notes (indexes, TTLs, geo)	8
7. Backend route inventory (all 20+ modules)	9
8. Notable engineering patterns	10
9. Things worth cleaning up	11

1 — Codebase Map

The repository is one monorepo with three independently-runable pieces, all talking to the same backend API:

Folder	What it is	Language / stack
<code>claimitapporg/lib</code>	The Flutter mobile app. Organised feature-by-feature under <code>features/</code> (auth, claims, bill_reader, shops, deals, select, classifieds, etc.), each with its own <code>screens/</code> , <code>providers/</code> , and <code>models/</code> . Shared pieces (network client, router, theme) live under <code>core/</code> .	Dart / Flutter, Provider for state, GoRouter for navigation, Dio for HTTP
<code>fastapi/backend/app</code>	The API both the app and the website talk to. One Python file per feature area under <code>routes/</code> (20 modules), Pydantic models under <code>models/</code> , and shared helpers (auth, S3, OTP, push, email) under <code>utils/</code> .	Python / FastAPI, Motor (async MongoDB), JWT
<code>claimit_web_org/frontend</code>	The React web portal (Advertiser / Sales / Shop Owner dashboards), calling the same backend.	React 18, Vite

Everything in the app and the website ultimately funnels through one place: `fastapi/backend/app/main.py`, which wires up 20 route modules onto a single FastAPI instance backed by one MongoDB database (`claimit_db`). The rest of this document walks through that code directly, not a summary of it.

2 — Step by Step: Backend Startup

File: `fastapi/backend/app/main.py` + `database.py`. This is what actually happens, in order, every time the API process starts (locally, or cold-starts on Vercel):

- 1 **Environment loads.** `load_dotenv()` reads `backend/.env` into `os.environ` before anything else — this is where the Mongo URL, JWT secret, Twilio/S3/Razorpay/Firebase keys all come from (via `config.py`'s `Settings`, a Pydantic `BaseSettings` class, cached with `@lru_cache` so it's parsed once).
- 2 **FastAPI's `lifespan` context manager runs on startup** (not simple `@app.on_event` — this is the modern async context-manager form). It calls `connect_db()` and waits for it before serving any requests.
- 3 **`connect_db()` opens the Motor client** against MongoDB Atlas, then runs a long list of index-creation and self-healing calls — see Section 6 for the interesting ones (sparse unique indexes, TTL indexes, a 2dsphere geo index with a one-time backfill).
- 4 **A data-normalization pass runs** directly inside `lifespan()`: it regex-matches every shop's `shop_type` field and force-corrects `has_rewards` / `has_redeem` flags so older, inconsistently-saved shops don't appear in both the Reward and Redeem lists. It's wrapped in try/except so a bad record can never block the whole API from booting.
- 5 **A background task starts:** `asyncio.create_task(daily_push_loop())` — a loop that fires one engagement push notification per day at a random time between 10:00–20:00 IST, running for the whole lifetime of the process.
- 6 **The FastAPI app object is built** with Swagger at `/docs` and ReDoc at `/redoc`, then CORS middleware is added — currently `allow_origins=["*"]` (open to any origin; flagged in Section 9).
- 7 **Static file serving is mounted** at `/uploads` for local files (avatars/claims/shop_images folders are auto-created) — this is the fallback used before/alongside the newer S3-backed storage.
- 8 **20 routers are registered** with `app.include_router(...)` — auth, users, claims, notifications, dashboard, policies, locations, deals, shops, rewards, redeem, reels, classifieds, admin, bill, banners, advertiser, feedback, payments, learn, select. Each is a self-contained `APIRouter` with its own `prefix`, so e.g. every route in `claims.py` automatically lives under `/claims/...`.
- 9 **A catch-all exception handler** is registered last — any unhandled exception anywhere in the app returns a generic `500 {"detail": "Internal server error"}` instead of leaking a stack trace to the client.



3 — Step by Step: How the App Talks to the Backend

File: `claimitapporg/lib/core/network/api_client.dart`. Every single network call in the Flutter app goes through one singleton, `ApiClient`, wrapping a Dio HTTP client.

- 1 **Singleton construction.** `ApiClient._internal()` runs once for the whole app lifetime (factory constructor always returns the same instance), configuring the base URL, timeouts, and default headers.
- 2 `validateStatus: (status) => status < 500` — this is a deliberate choice (and, per the build log, the fix for a real bug: a 401 redirect loop on Flutter web). Dio normally throws on any 4xx/5xx; here it only throws on 5xx, so the app's provider layer can read a 400/401/404's error body and show the real message instead of catching a generic exception.
- 3 **Request interceptor** — `onRequest` : before every call, unless the path starts with `/auth/` (so login/register/refresh never send a stale token), it reads the saved access token from `flutter_secure_storage` (Android EncryptedSharedPreferences / iOS Keychain) and attaches `Authorization: Bearer <token>`.
- 4 **Error interceptor** — `onError` : if a response comes back `401` and the request hasn't already been retried, it calls `_refreshToken()`, which POSTs the stored refresh token to `/auth/refresh`. If that succeeds, the new access token is saved, the original request is silently replayed with the new token (`_retried` flag prevents an infinite loop), and the caller never sees the 401. If refresh fails, tokens are wiped and the app's router redirects to login.
- 5 **Everything else is a thin wrapper** — `get/post/put/patch/delete/uploadFile` all just call the underlying Dio instance, plus helpers to save/clear tokens and build an absolute URL from a relative one the backend returns (e.g. turning `/uploads/avatars/x.png` into a full URL for `Image.network`).

Net effect: every screen in the app can call `ApiClient().get('/claims')` without knowing or caring whether the access token just expired — the interceptor pair handles authentication and silent refresh transparently for the whole app.

4 — Step by Step: Sign-in / OTP Flow, End to End

Files: `otp_screen.dart` / `auth_provider.dart` (app) → `routes/auth.py` (backend). This traces one real login from tap to dashboard.

- 1 User types a phone number (or email) and taps Continue. App calls `POST /auth/send-otp` with `{"phone": identifier, "mode": "login" | "register"}`.
- 2 Backend's `send_otp()` looks the identifier up with `_find_user()`, which checks for `"@"` in the string to decide whether to query `users.email` or `users.phone` — **the same field, "phone", carries either a mobile number or an email address**, kept for backward-compatibility with the app's existing form field.
- 3 Mode is enforced server-side: `login` mode 404s if no account exists ("register first"); `register` mode 409s if one already does ("login instead"). This prevents the classic bug where "forgot password" style flows silently create duplicate accounts.
- 4 A 6-digit OTP is generated, stored in `otp_store` (TTL-indexed — auto-expires), and `send_otp_sms()` attempts delivery. Delivery failure is deliberately swallowed (`try/except`, just logged) — the OTP is already stored, so the user can still enter it if delivery is slow, and a flaky SMS provider can never turn into a 500 for the user.
- 5 User enters the code. App calls `POST /auth/verify-otp` with the identifier, the code, and (for registration) the name typed on the register screen.
- 6 `verify_otp_endpoint()` validates the OTP, then either finds the existing user or — only in `register` mode — auto-creates one via `_auto_create_user()`. That helper deliberately **omits** the phone/email field entirely rather than storing `null` / `""`, because MongoDB's sparse unique index only skips documents where the field is *absent* — a stored empty string would still collide on the next email-only signup (this exact bug is what `_clean_legacy_empty_credentials()` in `database.py` exists to repair for old rows).
- 7 A JWT access token (`create_access_token`, 24 h expiry) and refresh token (`create_refresh_token`, 30-day expiry) are issued, both encoding `{"sub": user_id}` and signed with the app's `secret_key`.
- 8 If the app passed an FCM push token in the same request, it's upserted into `fcmtokens` right in this call — no separate round-trip needed to start receiving push notifications.
- 9 A "✅ Login Successful" notification is written via `notify_user()` — best-effort, its own errors are swallowed so a push-delivery outage can never fail the login itself.
- 10 Response returns both tokens + the serialized user object. The app's `AuthProvider` saves them via `ApiClient().saveTokens()` into secure storage and updates its `ChangeNotifier` state, which `GoRouter`'s redirect logic picks up to route the user to `/home`.

5 — Step by Step: Filing a Claim, End to End

Files: `new_claim_screen.dart` / `upload_documents_screen.dart` (app) → `routes/claims.py` (backend).

- 1 User fills the multi-step new-claim form (type, policy number, description, incident date, claimed amount) and submits. App calls `POST /claims/create`, JWT attached automatically by the interceptor from Section 3.
- 2 `get_current_user()` (a FastAPI `Depends`) runs first: decodes the JWT, confirms its type is `"access"` (not a refresh token misused as one), and loads the user from MongoDB by the `sub` claim. If any of that fails, the route body never executes — a 401 is returned automatically.
- 3 `create_claim()` generates a human-readable `claim_number`, builds the claim document with `status: "pending"` and a one-entry `timeline` array (`{"status": "pending", "message": "...", "timestamp": now}`) — this timeline array is what powers the claim-detail screen's status history UI.
- 4 The document is inserted into `claims`, and a notification ("Claim Submitted") is written for the user in the same request — synchronously, unlike the login notification, so it's guaranteed to exist before the response returns.
- 5 Back in the app, the claim-detail screen prompts for evidence. Each file goes to `POST /claims/{id}/documents` as multipart form data.
- 6 The backend re-checks the claim belongs to the calling user, validates `content_type` against an allow-list (PDF/JPEG/PNG/DOC/DOCX) and enforces `max_file_size_mb` from settings, then streams the bytes to S3 via `upload_bytes()` under `claims/{claim_id}/...` — **note this is a genuine S3 upload, not the local `/uploads` static folder** mentioned in Section 2; claims documents have already been migrated off local disk.
- 7 Only the S3 key is pushed onto the claim's `documents` array in MongoDB — never a permanent public URL. Every time a claim is read back (`get_claims` / `get_claim`), `_resolve_document_urls()` calls `generate_presigned_url()` fresh for each key, so documents stay private and links expire rather than being permanently public.
- 8 Later, an admin/staff action can call `PATCH /claims/{id}/status`. This appends a new timeline event, maps the new status to a canned customer-facing message (e.g. `approved` → "Approved amount: Rs.X"), writes a notification with a status-specific type (`success` / `error` / `info`), and returns the fully updated claim — the app's claim-detail screen reflects the new timeline entry on next load.

6 — Database Design Notes

File: `fastapi/backend/app/database.py`. A few choices in here are worth understanding, not just listing:

What	Why it's built this way
Sparse unique indexes on <code>users.phone</code> and <code>users.email</code>	A user can sign up with just a phone or just an email. A normal unique index would reject the second phone-less user because both would have the same missing value; <code>sparse=True</code> tells MongoDB to skip documents where the field doesn't exist at all rather than treating "missing" as a value to de-duplicate.
Self-healing index repair at boot (<code>_ensure_phone_index_sparse</code> , <code>_ensure_email_index</code>)	MongoDB's <code>create_index()</code> is a no-op if an index with that name already exists — even with different options. So on every boot the code explicitly inspects the live index's options and drops+recreates it if it's the old, wrong shape. This is how a schema fix (adding <code>sparse</code>) gets safely rolled out without a manual migration step.
<code>_clean_legacy_empty_credentials()</code>	Removes any leftover <code>phone: ""</code> / <code>email: ""</code> fields from before the sparse-index fix — an empty string isn't "missing" to MongoDB, so old rows would still trip the unique constraint without this cleanup.
<code>otp_store</code> TTL index (<code>expireAfterSeconds=0</code>)	Each OTP document carries its own <code>expires_at</code> ; MongoDB automatically deletes it once that time passes — no cron job needed to clean up old codes.
<code>bill_scans</code> TTL index (24 h) vs. permanent <code>bill_history</code>	Two different collections for the same event: <code>bill_scans</code> is short-lived and exists only to detect a duplicate scan of the same bill within 24 hours (compound unique index on <code>user_id</code> + <code>dup_key</code>); <code>bill_history</code> has no TTL and is the permanent record shown in the Rewards Wallet screen.
<code>shops</code> 2dsphere geo index + one-time backfill	<code>_backfill_shop_geo()</code> runs on every boot but is idempotent — it only touches shops that have <code>lat</code> / <code>lng</code> but no GeoJSON <code>geo</code> point yet, converting them so <code>/shops/nearby</code> can use a real MongoDB <code>\$geoNear</code> query instead of loading every shop into Python and computing distances by hand.
<code>select_conversations</code> unique compound index on (<code>customer_id</code> , <code>professional_id</code>)	Guarantees exactly one chat thread per customer/professional pair for the Claimit Select booking feature — a second "start chat" call finds the existing thread instead of creating a duplicate.

7 — Backend Route Inventory

All 20 route modules registered in `main.py`, each an independent `APIRouter` :

Module	Covers
<code>auth.py</code>	OTP send/verify, social login (Google/Facebook), token refresh, logout — see Section 4.
<code>users.py</code>	Profile read/update, avatar upload.
<code>claims.py</code>	Claims CRUD, document upload, status timeline — see Section 5.
<code>notifications.py</code>	List/read notifications.
<code>dashboard.py</code>	Aggregated home-screen stats.
<code>policies.py</code>	Insurance policy CRUD.
<code>locations.py</code>	State/pincode/geo reference data.
<code>deals.py</code>	Deal listing and detail.
<code>shops.py</code>	Shop listing (category/nearby/premium-first), shop detail, registration — largest route file after <code>bill.py</code> .
<code>rewards.py</code>	Reward point rules per shop/category.
<code>redeem.py</code>	Point redemption eligibility + processing.
<code>reels.py</code>	Short-video feed listing.
<code>classifieds.py</code>	Buy/sell listings CRUD.
<code>admin.py</code>	Admin-only aggregate stats and moderation actions.
<code>bill.py</code>	Bill-scan submission, reward calculation, wallet/history — the single largest route file (36 KB).
<code>banners.py</code>	Home-screen ad banners.
<code>advertiser.py</code>	Advertiser-portal campaign management (web).
<code>feedback.py</code>	Feedback/complaints ticket system.
<code>payments.py</code>	Razorpay payment endpoints.
<code>learn.py</code>	"Learn Claimit" video lessons (admin CRUD + public list + likes).
<code>select.py</code>	Claimit Select — professional listings, bookings, and chat (32 KB, second-largest route file — this module alone covers registration, browsing, booking, and messaging).

Shared, cross-cutting logic lives in `app/utils/`: `auth.py` (JWT), `otp.py` , `s3.py` (presigned uploads/downloads), `fcm.py` + `daily_push.py` (push notifications), `cashfree.py` (a second payment provider alongside Razorpay), `welcome_email.py` , and `notify.py` (the shared best-effort in-app notification writer used across most routes).

8 — Notable Engineering Patterns

- ✓ **Best-effort side effects never fail the main action.** OTP SMS delivery, push notifications, and the shop-flag normalization pass are all wrapped so their failure is logged, not raised — the pattern repeats across `auth.py`, `claims.py`, and `main.py`.
- ✓ **Presigned URLs, not stored public links.** Both claim documents and (per the shop model) shop images appear to store only the S3 key, resolving a fresh signed URL on every read — safer than baking permanent public URLs into the database.
- ✓ **Self-healing migrations at boot instead of one-off scripts.** Index shape fixes and the shop-flag normalization both run idempotently every time the server starts, rather than requiring someone to remember to run a migration script during deployment.
- ✓ **One identifier field, two meanings.** The `"phone"` field in login/OTP requests accepts a phone number or an email address, disambiguated with a simple `"@" in identifier` check — a pragmatic way to avoid a breaking API/app change while adding email login.
- ✓ **Transparent token refresh on the client.** The Dio interceptor pair in `ApiClient` means no individual screen or provider needs to think about expired tokens — it's handled once, centrally.

9 — Things Worth Cleaning Up

CORS is wide open: `allow_origins=["*"]` in `main.py`. Already flagged in the architecture doc's production checklist — fine for development, should be locked to the real app/website domains before launch.

Two payment providers wired in parallel: both `cashfree.py` and a Razorpay config/route exist. Worth confirming which one is actually going live so the other isn't left half-configured in production.

Many `.bak_<timestamp>` files sitting next to live code (e.g. `shops.py.bak_20260817_132536`, `home_screen.dart.bak_20260818_143532`, `app_router.dart.bak_20260819_111359` — dozens across both the backend and the app). These look like manual backups made before edits; since the project is already in Git, they're redundant and worth deleting so the working tree matches what's actually deployed.

Compiled `__pycache__/*.pyc` and local `uploads/` sample images are present in the working folder alongside source — normal for local dev, just confirm they're excluded via `.gitignore` so they don't get committed.

`generate_assets.py`, `chk_cats.py`, `chk_shops.py`, `seed.py` / `seed_select.py`, `twilio_test.py` are one-off/dev scripts living at the project root and inside `backend/` — useful to keep, but worth a short README note on what each does and when it's safe to re-run (some, like seed scripts, likely shouldn't be re-run against production data).